

Practical Week 9

Aim

To implement a file-copy program using Linux system calls and compare it with a standard C library file-copy program.

Program:

```
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical9A.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical9A.c -o Practical9A
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical9A
Usage: ./Practical9A source destination
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical9B.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical9B.c -o Practical9B
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical9B
Usage: ./Practical9B source destination
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical9C.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical9C.c -o Practical9C
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical9C
input.txt: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical9B.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    FILE *source;
    FILE *destination;

    char buffer[1024];
    size_t bytesRead;

    if (argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");

    if (source == NULL)
```

```

saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical7A

PROCESS ADDRESS SPACE

Code address   : 0x636c899cc1e9
Global address : 0x636c899cf010
Static address : 0x636c899cf014
Heap address   : 0x636cb029b010
Stack address  : 0x7fff8cf2eb5c

Process ID: 1021
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical7B.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical7B.c -o Practical7B
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical7B
Process ID: 2664
Stack address: 0x7ffdc3c8fcfc
Heap address  : 0x6105bb7cf010

Process is running.
Open another terminal and execute:
cat /proc/2664/maps

Press Enter to exit...
^C
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat /proc/5678/maps
cat: /proc/5678/maps: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat /proc/2664/maps
cat: /proc/2664/maps: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat /proc/2664/maps
cat: /proc/2664/maps: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat /proc/1021/maps
cat: /proc/1021/maps: No such file or directory
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical7A.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_variable = 100;

static int static_variable = 200;

void sample_function() {
    printf("Inside sample function.\n");
}

int main() {

```

Output:

```
source = fopen(argv[1], "rb");

if (source == NULL)
{
    perror("fopen");
    return 1;
}

destination = fopen(argv[2], "wb");

if (destination == NULL)
{
    perror("fopen");
    fclose(source);
    return 1;
}

while ((bytesRead = fread(buffer, 1, sizeof(buffer), source)) > 0)
{
    fwrite(buffer, 1, bytesRead, destination);
}

fclose(source);
fclose(destination);

printf("File copied successfully using standard C library.\n");

return 0;
}
saikrishna_90156@DESKTOP-2JF9H5T:~$ cat Practical9C.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int inputFile;
    int outputFile;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile == -1)
    {
        perror("input.txt");
        return 1;
    }

    outputFile = open("output.txt",
```

Observation

- `open()` opens the file.
- `read()` reads data from the source file.
- `write()` writes data to the destination file.
- `close()` closes the files.
- The standard library version uses `fopen()`, `fread()`, `fwrite()`, and `fclose()`.
- Both programs successfully copy the file.
- `cmp` verifies that the copied files are identical.
- `time` can be used to compare their execution time.

Result

A file-copy utility was successfully implemented using low-level Linux system calls and compared with a standard C library implementation.

When you finish this, say **next** and we'll do **Practical Week 9 – Question 2: Standard Input/Output Redirection using `dup2()`**.

Aim

To understand file descriptors and implement input and output redirection using the Linux dup2() system call.

```
}
saikrishna_90156@DESKTOP-2JF9H5T:~$ cat Practical9A.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    if (argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source == -1)
    {
        perror("open");
        return 1;
    }

    destination = open(argv[2],
                       O_WRONLY | O_CREAT | O_TRUNC,
                       0644);

    if (destination == -1)
    {
        perror("open");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");
}
```

```

saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical9C.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int inputFile;
    int outputFile;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile == -1)
    {
        perror("input.txt");
        return 1;
    }

    outputFile = open("output.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (outputFile == -1)
    {
        perror("output.txt");
        close(inputFile);
        return 1;
    }

    /* Redirect standard input */
    if (dup2(inputFile, STDIN_FILENO) == -1)
    {
        perror("dup2 input");
        return 1;
    }

    /* Redirect standard output */
    if (dup2(outputFile, STDOUT_FILENO) == -1)
    {
        perror("dup2 output");
        return 1;
    }

    close(inputFile);
    close(outputFile);

    printf("This message has been redirected to output.txt\n");
}

```

Program:

Final Result

File copying using low-level system calls, comparison with standard library functions, and standard I/O redirection using dup2() were successfully

implemented and verified.

```
}
saikrishna_90156@DESKTOP-2JF9H5T:~$ cat Practical9A.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    if (argc != 3)
    {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source == -1)
    {
        perror("open");
        return 1;
    }

    destination = open(argv[2],
                       O_WRONLY | O_CREAT | O_TRUNC,
                       0644);

    if (destination == -1)
    {
        perror("open");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");
}
```